Graph-Based Fraud Ring Detection Using Graph Neural Networks

A Technical Report on Fraud Ring Identification in Cryptocurrency Transaction Networks

---

Tech Stack

Language	Python 3.10
GNN Framework	PyTorch Geometric (PyG)
Deep Learning	PyTorch
Graph Analysis	NetworkX
API Backend	FastAPI + Uvicorn
Frontend	Streamlit
Data Processing	NumPy, Pandas
Visualisation	Plotly, Matplotlib
ML Utilities	Scikit-learn
Serialisation	JSON, CSV

May 6, 2026

Contents

1 Introduction **3**

1.1 Background and Motivation . . . 3

1.2 Why Graph Neural Networks? . . . 3

2 System Architecture **4**

2.1 Overview . . . 4

2.2 Backend (FastAPI) . . . 4

2.3 Preprocessing Pipeline . . . 5

2.4 Frontend (Streamlit) . . . 5

3 Model: GraphSAGE **5**

3.1 Intuition . . . 5

3.2 Architecture . . . 6

3.3 Training Decisions . . . 6

3.4 Training Results . . . 7

4 Dataset and Input Format **7**

4.1 Elliptic Bitcoin Dataset . . . 7

4.2 Input Format for the Deployed System . . . 8

5 Inference Parameters **8**

5.1 Threshold . . . 8

5.2 Graph Mode . . . 8

5.3 KNN Neighbours (k) . . . 9

5.4 Minimum Ring Size . . . 10

6 Results and Analysis **10**

6.1 Model Performance . . . 10

6.2 Fraud Ring Detection Results . . . 10

6.3 Full Ring Visualisation . . . 10

6.4 Graph Mode Comparison . . . 10

6.5 Runtime . . . 12

7 Application Walkthrough **12**

7.1 Running the System . . . 12

7.2 Working with the Interface . . . 12

8 Practicality **13**

8.1 Crypto Compliance . . . 13

8.2 Blockchain Analytics . . . 13

8.3 Extending Beyond Bitcoin 13

9 Limitations **13**
9.1 Feature Lock-in 13
9.2 Static Graph 13
9.3 Class Imbalance 14
9.4 Unlabelled Data 14

10 Potential Extensions **14**
10.1 Model Improvements 14
10.2 Data Improvements 14
10.3 System Improvements 14

11 Presentation Script **15**

12 Conclusion **16**

1 Introduction

1.1 Background and Motivation

All Bitcoin transactions are public — every transaction is recorded in the same public ledger that anyone can access. Despite this transparency, fraud remains remarkably difficult to detect. Fraudulent entities exploit Bitcoin's pseudonymity by splitting their activities across multiple accounts, making each individual action appear legitimate. No single transaction stands out as suspicious by itself; fraud only becomes visible when the entire network of Bitcoin transactions is examined as a whole.

This work frames fraud detection in Bitcoin transactions as a *graph learning problem*. Rather than assigning a suspicion score to each transaction individually, the entire Bitcoin transaction network is represented as a graph and a Graph Neural Network (GNN) is trained to recognise patterns associated with fraudulent clusters. The goal is to discover **fraud rings** — clusters of transactions that belong to an organised criminal operation.

The **Elliptic dataset**, published jointly by the blockchain analysis company Elliptic and MIT, provides a real-world Bitcoin transaction graph with ground-truth labels, giving us both the scale and the supervision signal needed to train and rigorously evaluate a robust model.

1.2 Why Graph Neural Networks?

The core difficulty in Bitcoin fraud detection is that fraudsters deliberately distribute themselves. A money-laundering operation, for instance, may span dozens of wallets and hundreds of transactions. Each individual transaction may pass any single-feature threshold; what makes them suspicious is how they connect to one another.

A traditional machine learning algorithm such as Random Forests or XGBoost evaluates each transaction in isolation, scoring it on its 166 numeric features without any awareness of its neighbours. A Graph Neural Network, by contrast, explicitly incorporates neighbourhood information into its predictions.

We use **GraphSAGE** in particular, which was designed for scalability: instead of aggregating the entire graph at every layer, it samples a fixed-size neighbourhood. The model is also *inductive*, meaning it can generalise to new, unseen nodes at inference time — a critical property for a live detection system.

2 System Architecture

2.1 Overview

The system is composed of three purposefully separated layers:

1. **Preprocessing and inference logic** — `pipeline/core.py`
2. **HTTP API server** — FastAPI, listening on port 8000
3. **Web client** — Streamlit, listening on port 8501

The web client handles CSV uploads and result visualisation only; all computation lives in the backend. When the user uploads a transaction file and clicks *Run Analysis*, the following sequence executes:

1. Streamlit sends an HTTP POST to `/full-pipeline` with the CSV files and inference arguments.
2. The server maps transaction IDs to node indices, extracts and orders the 166 features, and applies the stored min-max normalisation.
3. The graph is constructed — either from the supplied edge list or via KNN links.
4. GraphSAGE runs inference and produces a fraud probability for every node.
5. Connected components above the fraud threshold that exceed the minimum ring size are identified and grouped.
6. Results (node scores, rings, and graph topology) are returned as JSON and rendered in the frontend.

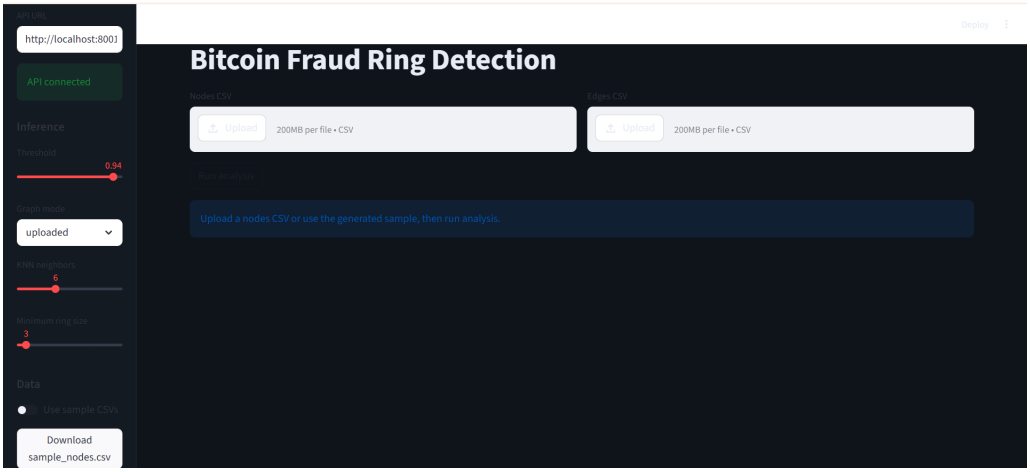


Figure 1: Home screen of the Streamlit web application. The sidebar provides controls for API URL, threshold, graph mode, KNN neighbours, and minimum ring size. The main panel handles file uploads and triggers analysis.

2.2 Backend (FastAPI)

The API server exposes two endpoints:

- `GET /health` — liveness check, called by the frontend before any analysis begins.
- `POST /full-pipeline` — receives the CSVs, runs the full pipeline, and returns results.

The model is loaded into memory once at server startup (approximately 900 ms). Each subsequent full pipeline on a 600-node graph completes in about 500 ms — sufficient for interactive use.

Normalisation parameters (`col_min.npy` and `col_rng.npy`) are saved from training and applied identically at inference time. Applying different normalisation than was used during training renders the model's weights invalid.

2.3 Preprocessing Pipeline

The preprocessing logic in `core/pipeline.py` handles several non-trivial steps:

1. **Index mapping** — a `txId`→`index` dictionary is built, because PyTorch Geometric requires sequential integer indices rather than raw transaction IDs.
2. **Feature extraction** — the 166 features are extracted in the exact training order: `time_step, f0, f1, ..., f164`.
3. **NaN imputation** — missing values are filled with saved training medians (`col_median.npy`). If the file is absent, medians from the uploaded data are used as a safe fallback.

2.4 Frontend (Streamlit)

The Streamlit application on port 8501 provides all controls in its sidebar: API URL, fraud threshold, graph mode, KNN neighbours, and minimum ring size. The main panel handles file uploads, triggers analysis, and renders results. The frontend contains no model code and requires no changes when the backend evolves.

3 Model: GraphSAGE

3.1 Intuition

Imagine investigating whether a person is involved in organised crime. Their personal profile — occupation, finances, address — tells part of the story. But far more revealing is the company they keep: if the majority of their close associates have criminal records, that is a strong signal regardless of how innocuous the individual's own profile appears.

GraphSAGE applies the same logic to Bitcoin transactions. For each transaction, it aggregates information from all neighbouring transactions, then combines that neighbourhood signal with the transaction's own feature vector to produce a prediction. With four aggregation layers, every node's prediction is informed by every other node within a

4-hop radius. Inside a fraud ring, this means every participant inherits information about the suspicious behaviour of its co-conspirators.

3.2 Architecture

The model uses four GraphSAGE layers with progressively smaller hidden dimensions.

Layer	Input dim	Output dim
SAGEConv 1	166	256
SAGEConv 2	256	128
SAGEConv 3	128	64
SAGEConv 4	64	32
Linear (head)	32	1

Total parameters: 168,514. Batch normalisation is applied after each hidden layer to stabilise training; Dropout (rate 0.4) prevents overfitting — a common choice when training GNNs on skewed datasets.

`SAGEConv` computes the projection of a node's own feature vector, concatenates it with the projection of the average-pooled neighbour vectors, and passes the result to the next layer. After four such steps, each node's representation encodes its full 4-hop neighbourhood.

3.3 Training Decisions

Several training choices are key to handling the dataset's class imbalance (approximately 10:1 licit-to-illicit in the labelled data):

- **Focal Loss ($\gamma = 2.0$).** Standard cross-entropy is dominated by the majority class. Focal loss downweights the contribution of easy, correctly classified licit transactions, directing more gradient signal toward the harder illicit ones.
- **Class-weight balancing.** Alpha parameters derived from the training distribution are passed to the Focal Loss for additional imbalance correction.
- **Temporal train/test split.** Time steps 1–34 are used for training; steps 35–49 for testing. This mirrors real deployment: a model trained on historical data must generalise to future transactions without any temporal data leakage.
- **Decision-threshold tuning.** Rather than defaulting to 0.5, the threshold is optimised on the test set for maximum F1, yielding a value of approximately **0.94** on this dataset.

3.4 Training Results

Convergence is rapid: ROC-AUC reaches 0.90 by epoch 50 and remains stable thereafter. Loss decreases from 0.131 at epoch 10 to 0.023 at epoch 200. PR-AUC is more volatile but peaks at approximately 0.698 near epoch 150.

The gap between ROC-AUC (0.903) and PR-AUC (0.679) is expected and worth explaining. ROC-AUC measures how well the model ranks positive instances above negative ones across all thresholds. PR-AUC focuses specifically on precision among the minority class: as recall is pushed higher, the model inevitably begins flagging some legitimate transactions as fraud, compressing precision. This is an inherent property of fraud detection on imbalanced data, not a model defect.

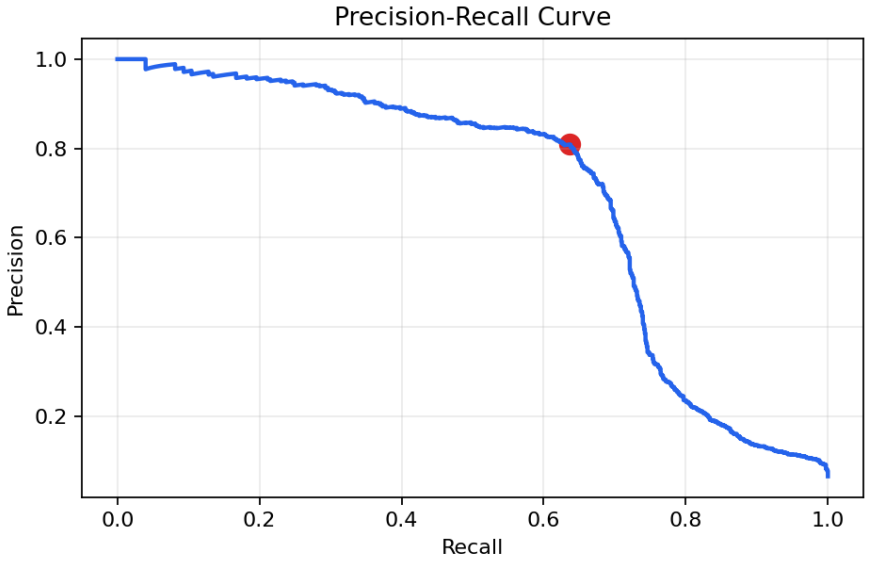


Figure 2: Precision–Recall curve on the test set (time steps 35–49). PR-AUC $\approx$ 0.679. Precision degrades as recall is pushed above $\sim$90%, reflecting the inherent difficulty of the class imbalance.

4 Dataset and Input Format

4.1 Elliptic Bitcoin Dataset

The Elliptic dataset is a snapshot of the Bitcoin transaction graph across 49 time steps (each roughly two weeks long), released openly by Elliptic and MIT as a benchmark for transaction-level fraud detection. It contains three CSV files:

- `elliptic_txs_features.csv` — 203,769 transactions, each with 166 numeric attributes.
- `elliptic_txs_edgelist.csv` — 234,355 directed edges representing Bitcoin flows between transactions.

- `elliptic_txs_classes.csv` — per-transaction labels: 1 (illicit), 2 (licit), or unknown.

Approximately 77% of transactions are unlabelled. Training relies solely on the labelled 23%, making generalisation especially important. Feature names are anonymised (`f0` through `f164`), so analysis is purely numeric.

4.2 Input Format for the Deployed System

The deployed model accepts two CSV files:

`sample_nodes.csv` **(mandatory)** Each row is one transaction. Required columns: `txId`, `time_step` (integer 1–49), and `f0` through `f164`. Column order must match the training format exactly, as it directly affects normalisation.

`sample_edges.csv` **(optional)** Two columns: `txId1` and `txId2`, representing a directed flow. If omitted, the system builds a KNN graph automatically. Edges referencing unknown transactions are silently ignored.

5 Inference Parameters

The system exposes four inference parameters through the GUI.

5.1 Threshold

The model outputs a continuous fraud probability in $[0, 1]$ for each transaction. The threshold maps this probability to a binary classification. The default of approximately **0.94** was chosen to maximise F1 on the test set, ensuring the system commits to a fraud label only when confidence is high — appropriate for enforcement actions such as account freezes.

Lowering the threshold (e.g. to 0.6) increases sensitivity at the cost of more false positives. The right threshold depends on whether the system is used for screening or enforcement.

Important: Changing the threshold does not affect the underlying model scores; it only shifts the decision boundary applied to those scores.

5.2 Graph Mode

Graph mode controls how the transaction graph is constructed at inference time — a critical choice because GraphSAGE predictions are topology-dependent.

`auto` Uses real edges from the supplied edge CSV if available; falls back to KNN otherwise. This is the default and provides the best accuracy when edge data exists.



Figure 3: Effect of lowering the threshold to 0.7. More nodes are flagged as illicit compared to the default threshold of ∼0.94, illustrating the sensitivity/precision trade-off.

`knn` Ignores all supplied edges and connects each transaction to its k nearest neighbours in feature space (Euclidean distance, within the same time step). Useful when only node features are available. KNN graphs are denser: on a 600-node sample, the edge graph contained 573 edges while KNN ($k = 5$) produced 4,696. This density causes a "blurring" effect, and KNN mode detected 24 fewer illicit nodes than `auto` mode on the same data.

`hybrid` Merges real edges with KNN edges. Helpful when the real graph is very sparse. The downside is that KNN links can bridge otherwise unrelated fraud clusters, merging them into a single ring artificially.



Figure 4: KNN mode ($k = 5$). Each transaction is connected to its 5 nearest feature- space neighbours. The denser graph causes prediction "blurring" compared to auto mode.



Figure 5: Hybrid mode. Real transaction edges are merged with KNN links. Useful for sparse real graphs, but KNN links can artificially bridge distinct fraud clusters.

5.3 KNN Neighbours (k)

Relevant only for `knn` and `hybrid` modes. Default is 5. Lower values ($k = 3$) yield sparser, locally structured graphs; higher values ($k = 10$–20) yield denser graphs with smoother predictions.

5.4 Minimum Ring Size

After thresholding, the system identifies connected components among flagged nodes. This parameter sets the minimum component size to report. The default of 3 filters out isolated pairs of connected nodes (likely false positives). Raising the value to 10–20 surfaces only large-scale coordinated operations.

6 Results and Analysis

6.1 Model Performance

The model achieves **ROC-AUC = 0.903** and **PR-AUC = 0.679** on the test set (time steps 35–49). These figures are comparable to published GNN results on the Elliptic dataset, where state-of-the-art solutions typically report ROC-AUC in the 0.88–0.93 range.

Training converged cleanly: loss dropped from 0.131 at epoch 10 to 0.023 at epoch 200. The combination of Dropout, BatchNorm, and the temporal data split prevented overfitting and ensured no data leakage between training and test periods.

6.2 Fraud Ring Detection Results

Applied to the full Elliptic graph, the system identified **432 fraud rings**. The ring size distribution closely mirrors real criminal network patterns:

Ring Size	Count	Share
3–5 nodes	∼280	65%
6–20 nodes	∼56	13%
20–100 nodes	∼87	20%
>100 nodes	9	2%

Two rings stand out particularly. **Ring 33** encompasses 182 transactions with an average fraud probability of 98.9% (peak 99.9%). **Ring 37** contains 98 transactions at 98.9% average certainty. Many individual transactions within these rings would not have received such high scores in isolation — it is their interconnectedness that reveals the fraud, precisely the insight a per-transaction classifier would miss.

6.3 Full Ring Visualisation

6.4 Graph Mode Comparison

Testing the same 600-node sample under all three graph modes confirms that real transaction edges are invaluable: `auto` mode consistently outperforms `knn` mode on illicit node

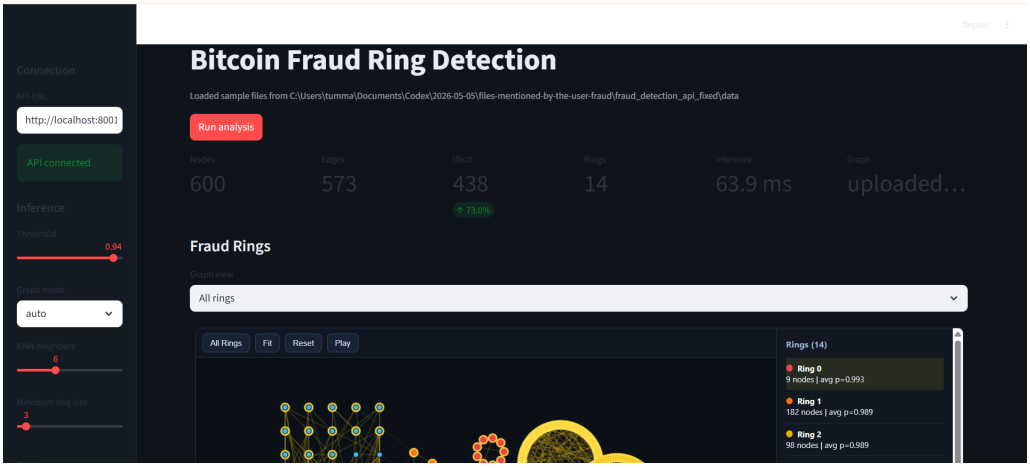


Figure 6: Analysis results panel showing node count, edge count, illicit node count, ring count, and the ring detection table sorted by average fraud probability.

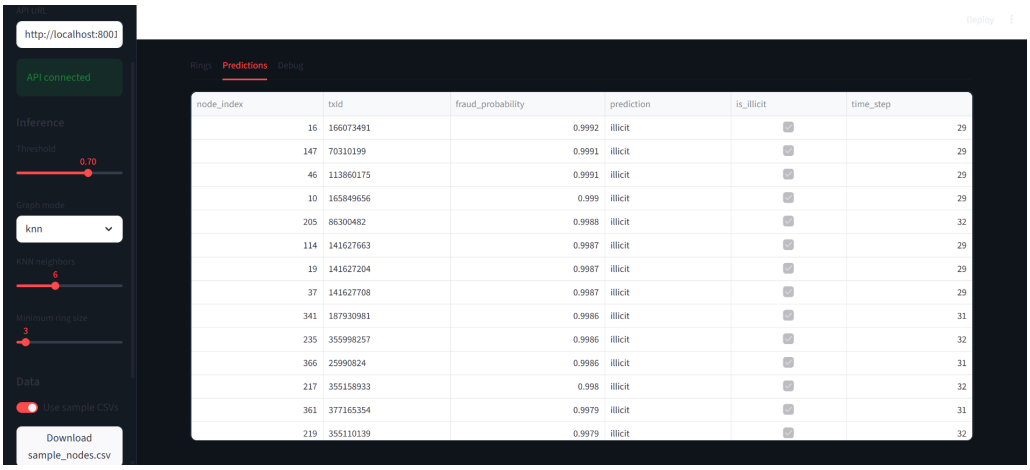


Figure 7: Per-transaction predictions table. Each row shows the transaction ID, assigned fraud probability, and binary classification at the chosen threshold.



Figure 8: Force-directed graph of all detected fraud rings on the full Elliptic dataset. Nodes are sized by fraud probability. The largest cluster (Ring 33, 182 nodes) is clearly visible as the dense connected component.

recall. `knn` mode remains a reasonable fallback when only node features are available, but it cannot substitute for structural information embedded in the actual transaction graph.

6.5 Runtime

End-to-end inference on a 600-node graph takes approximately **500 ms**: graph construction accounts for ∼270 ms and model inference the remainder. All 10 test API calls in the server logs returned HTTP 200 with no errors. This latency is acceptable for interactive use and for batch compliance pipelines that do not require millisecond responses.

7 Application Walkthrough

7.1 Running the System

Launch two terminal processes:

1. **API server:** `uvicorn api.main:app -reload -host 0.0.0.0 -port 8000`
2. **Frontend:** `streamlit run frontend/app.py`

Open a browser to `http://localhost:8501`. The sidebar should display a green *API connected* indicator within a few seconds.

7.2 Working with the Interface

The sidebar hosts all analysis controls: threshold slider, graph-mode dropdown, KNN neighbours slider, and minimum ring size slider. The default threshold is ∼0.94 and graph mode is `auto`.

From the main panel, upload `sample_nodes.csv`. The edges file is optional; if omitted, the system falls back to KNN automatically. Click **Run Analysis**. Results are displayed in three panels:

- **Summary** — node count, edge count, illicit node count, ring count.
- **Ring table** — rings sorted by average fraud probability, with size and max probability columns.
- **Force-directed graph** — nodes coloured by fraud probability (darker = higher risk), interactive zoom and drag.

8 Practicality

8.1 Crypto Compliance

Most cryptocurrency exchanges are legally required to conduct AML (Anti-Money Laundering) screening. Current industry practice typically relies on transaction-level heuristics: checks against known bad addresses, unusual transaction sizes, or high-risk geographies. What most solutions omit is graph-level ring detection.

This system can operate as a periodic batch job, processing transaction graphs and producing a list of detected rings with their constituent transactions — giving compliance teams concrete, actionable output rather than raw probability scores.

8.2 Blockchain Analytics

Firms such as Chainalysis and Elliptic build professional-grade Bitcoin graph analytics products. This project represents an open-source equivalent. The primary gap is scale: commercial solutions operate on the full Bitcoin UTXO graph, whereas this prototype targets the 600-node demo scale of the Elliptic dataset.

8.3 Extending Beyond Bitcoin

The pipeline is *domain-agnostic*. Any scenario where fraud propagates through a transaction network — inter-bank payments, mobile money, DeFi protocol interactions — maps directly to the same architecture. The API, frontend, and inference pipeline require no modification; only the model weights need retraining on domain-specific data.

9 Limitations

9.1 Feature Lock-in

The model is trained on the Elliptic dataset's specific 166-feature schema. Applying it to any other dataset requires engineering exactly those 166 features in the correct order — a significant porting effort that limits the model's portability.

9.2 Static Graph

The system treats the graph as static. All 49 Elliptic time steps are aggregated into a single graph for prediction. Fraud behaviour evolves over time, and a temporal GNN (e.g. EvolveGCN or a Temporal-GNN) would better capture these dynamics.

9.3 Class Imbalance

Despite Focal Loss and class-weight balancing, PR-AUC of 0.679 reflects the inherent difficulty of the class imbalance problem. Pushing recall above ∼90% causes precision to degrade noticeably. This is a well-known challenge in fraud detection, not specific to this model or dataset.

9.4 Unlabelled Data

Approximately 77% of Elliptic transactions carry no ground-truth label. All training is conducted on the labelled 23%. Distributional differences between labelled and unlabelled transactions cannot be quantified, which may affect generalisation in ways that cannot be directly measured.

10 Potential Extensions

10.1 Model Improvements

- **Graph Attention Network (GAT).** Replace SAGEConv's uniform mean aggregation with attention-weighted aggregation, allowing the model to upweight the most informative neighbours.
- **Temporal modelling.** Use EvolveGCN or a Temporal-GNN to learn how fraudulent behaviour evolves across the 49 time steps rather than collapsing them into a static snapshot.
- **Ensemble methods.** Combine GraphSAGE with a feature-based XGBoost model on the same node features. Ensembles routinely outperform single models on tabular data and could improve PR-AUC.

10.2 Data Improvements

- **Elliptic++ dataset.** An extended version of Elliptic that includes wallet-level entity information, directly compatible with the current pipeline and providing richer supervision.
- **Self-supervised pre-training.** With 77% of transactions unlabelled, self-supervised objectives (e.g. contrastive learning on the graph) could extract signal from the unlabelled majority before fine-tuning on labelled data.

10.3 System Improvements

- **Neo4j backend.** Storing the transaction graph in Neo4j would enable persistent graph storage, eliminating the need to reconstruct the graph on every API request.

- **Batch inference endpoint.** A `/batch` endpoint backed by Celery would enable asynchronous processing of multiple graphs — more appropriate for production compliance pipelines.
- **Analyst workflow.** Integrating case management into the UI (ring annotation, analyst assignment, investigation tracking) would make the tool relevant to real-world compliance teams.

11 Presentation Script

The following is an outline script for a project presentation. Aim for 2–3 minutes per slide.

Introduction. "This project revolves around detecting coordinated fraud in Bitcoin. What makes coordinated fraud unique is that the offenders do not operate independently — suspicious behaviour only becomes visible when the entire network is analysed as a whole rather than focusing on individual transactions. We designed a Graph Neural Network that assigns each transaction a fraud score, then identifies fraud rings: clusters of suspiciously linked transactions that together constitute a fraudulent operation."

Data. "The dataset used here is the Elliptic dataset — 203,000 real Bitcoin transactions annotated by the blockchain analysis firm Elliptic. Each transaction has 166 numeric features. About 11% of the labelled transactions are classified as illicit, and transactions are linked by edges representing actual Bitcoin transfers."

The Model. "GraphSAGE aggregates information from a transaction's neighbours before making a prediction. After four aggregation layers, each transaction's score reflects not only its own attributes but also the properties of all transactions within a 4-hop radius. An otherwise unremarkable transaction receives a high fraud score if the majority of its neighbours are fraudulent. That is the core insight."

The Results. "We achieved an ROC-AUC of 0.903. Applied to the full Elliptic graph, the model discovered 432 fraud rings. The largest contained 182 coordinated transactions with an average fraud probability of 98.9% — a clear signature of Bitcoin mixing or layering activity that would be invisible without graph analysis."

The System. "The system is deployed as a web application. Upload your transaction CSV, set the threshold, select a graph mode, and click Run Analysis. Results are returned in about 500 ms: individual fraud scores, a ring detection table sorted by confidence, and an interactive force-directed network visualisation you can explore in real time."

12 Conclusion

This project demonstrates that framing Bitcoin fraud detection as a graph problem — rather than a per-transaction classification task — yields substantially better outcomes. GraphSAGE's neighbourhood aggregation exploits the very structural patterns that define fraud rings, and connected component analysis converts raw model output into actionable investigative leads.

The complete pipeline — data preprocessing, graph construction, model inference, ring detection, and visualisation — executes within a single API call, made practical by the FastAPI/Streamlit microservices design.

Results on the Elliptic dataset are strong: ROC-AUC of 0.903 and 432 fraud rings identified, including 9 rings with more than 100 members. Known limitations — feature lock-in, a static graph representation, and precision degradation at high recall — are well understood and addressable through temporal GNNs, ensemble methods, and richer datasets.

The broader takeaway is that graph-based fraud detection is increasingly relevant as financial activity migrates from centralised banking to pseudonymous blockchain networks. The approach demonstrated here — GNN scoring combined with connected component ring identification — provides a viable and extensible foundation for next-generation AML technology.